Detecting Item Transitions Across a Boundary from RFID Reads

Authors: Ajin Frank Justin
Sponsor: Senior Director, Machine Learning
GitHub: https://github.com/justin-aj/ds5500-capstone

Abstract

Inventory systems use RFID readers to record an item as received or shipped when its tag crosses a boundary like a doorway. The reader never sees the crossing, though. It only reports reads, and tags that turn back or sit parked nearby get read too. I'm building a classifier that decides, for each pass of a tag, whether it crossed, using reads from eight antennas in three groups around one boundary. My 19 features compare the antenna groups with each other (which groups heard the tag, their share of reads, the order and the signal gaps), so they don't depend on item speed. My logistic regression baseline, trained only on the first data collection, found 998 of 999 straight passes in a second, separate recording, with 16 false alarms out of 41,327 parked tags, and 24 false alarms out of 3,897 parked tags in a third. Turn-backs and outward crossings are still untested on new recordings.

1. Introduction

An RFID tag is a small battery-free sticker that stores an ID. A reader sends a radio signal through its antennas, one at a time, and every tag in range answers [1]. Each answer is a *read*: tag ID, antenna, timestamp and signal strength (RSSI, in dBm; -50 is strong, -80 weak). A read doesn't say where the tag is or which way it's moving.

Domain problem. Antennas around a doorway watch a boundary between outside and inside. An item crossing it means goods were received or shipped. But many tags are heard without crossing: they turn back, sit parked nearby or get picked up from a distance. A *false alarm* logs a crossing that never happened. A *missed crossing* leaves a real movement unrecorded. Their costs differ by site, so I measure both.

Analytic problem. The input is one *pass*: every read of one tag near the boundary, from all eight antennas. The output is **in**, **out** or **none**. None of my recordings have an outward crossing yet, so this report covers **in** vs. **none**. The antennas form one read zone (Figure 1): ports 13 and 15 before the boundary (*outside*), 9-12 at it (*middle*) and 14 and 16 after it (*inside*).

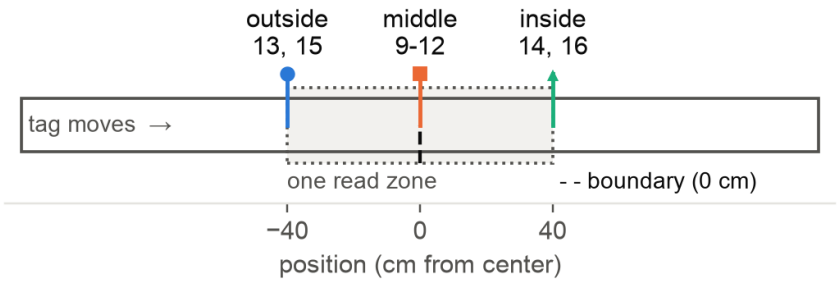


Figure 1. Antenna setup. Eight antennas in three groups form one read zone around one boundary. Numbers are antenna ports.

2. Methods

2.1 Data

All three collections use the same eight antennas (Table 1). The reader visits each port for about 30 ms, so one *round* of reads takes about 0.27 s. Tags move at 40 cm/s.

Collection	Size	Labels	Use
First	5.2M reads, 67,033 passes	known movement: 22,943 straight passes, 14,346 + 12,920 turn-backs (two types), 16,824 parked tags	train and test
Second	1.86M reads, 42,326 passes	999 tags recorded as carried through; the rest parked	test
Third	0.77M reads, 3,897 passes	all parked, two antenna configurations	false alarms

Table 1. The three data collections. In the second collection one pass is one tag in one iteration of the recording; in the third, one tag in one run.

2.2 Labels: the forward rule

In the first collection I know each pass's movement type. Turn-backs and parked tags are **none**. A straight pass is **in** only if its reads move forward: heard by the middle group and at least one end, outside never heard in a later round than the last middle read, and inside never heard in an earlier round than the first middle read. Out of 22,943 straight passes, 19,668 pass the rule. The other 3,275 (2,648 heard only by the middle group and 627 that look backward) did cross, so I kept them out of training as a test set, the *hard straight passes*. That leaves 63,758 labeled passes (19,668 **in**, 44,090 **none**).

2.3 Exploratory analysis

Figure 2 shows a typical pass of each type. A straight pass moves from outside to middle and is loudest at the middle. A type 2 turn-back reaches the inside group and comes back, so its order alone can look like a straight pass. A parked tag gives a flat, weak signal for 300 s, against about 7 s for a moving pass.

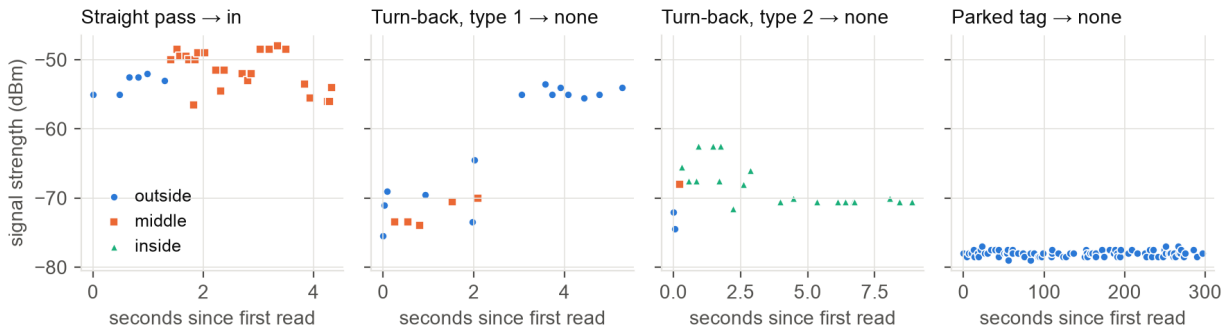


Figure 2. Signal strength of every read over time for one typical pass of each movement type in the first collection (read count nearest the median). Color and marker give the antenna group.

Two findings shaped the features (Figure 3). First, which groups heard a tag can't be the label: 38% of straight passes are never heard by the outside or inside group, in both collections. Second, tag strength flips what the reads mean. A strong tag heard outside and middle but never inside is a turn-back 88% of the time; a weak tag with the same reads crossed 94% of the time. The reader doesn't report tag strength, so my features compare antenna groups instead of counting reads.

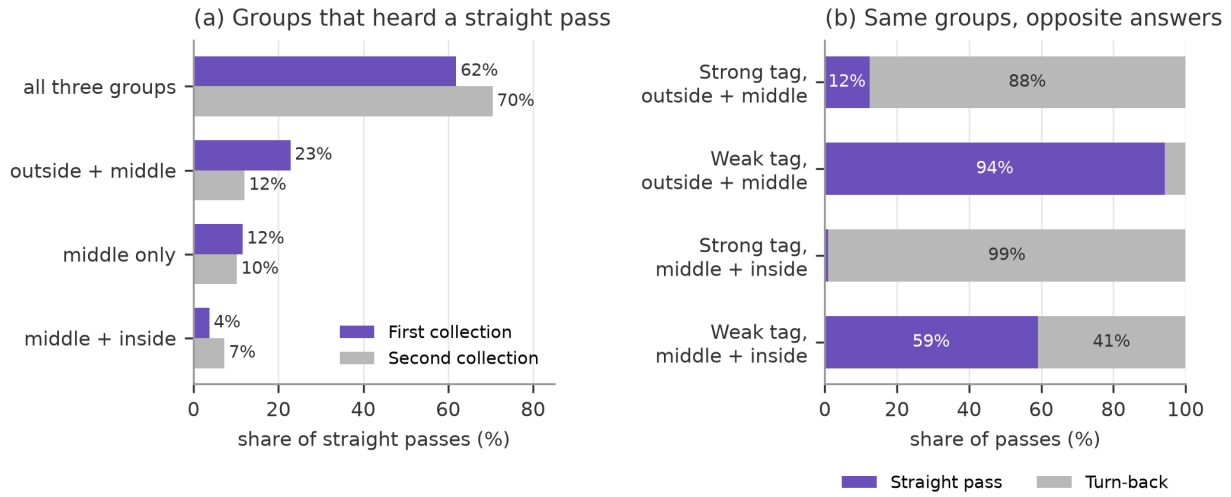


Figure 3. (a) Share of straight passes by which antenna groups heard the tag, first collection (n = 22,943) against the second (n = 999). (b) First-collection passes heard by only two groups, by tag strength: share of straight passes and turn-backs.

2.4 Features

Each pass becomes 19 features in five groups (Table 2; full list in Appendix A). Timing is a share of the pass, so speed doesn't change it. I dropped tag strength and angle (a reader doesn't report them) and pass length and raw read counts (they change with speed).

Feature group	Question it answers	Typical straight pass
Heard at all (3)	Did each antenna group hear the tag?	inside heard: yes
Share of reads (3)	What share of the reads came from each group?	middle share 77%
Order (9)	When was each group first heard, last heard and loudest, as a share of the pass?	inside first heard at 0.58
Signal gaps (3)	How much louder is the middle than the ends? Outside against inside? How much does the signal swing?	middle minus ends +10 dB
Middle coverage (1)	How many of the 4 middle antennas heard it?	4 of 4

Table 2. The 19 features in five groups, with a median value for a typical straight pass.

2.5 Model and evaluation

My baseline is logistic regression in scikit-learn [2]. Empty order features are filled with -1, features are standardized, classes get balanced weights and the threshold is 0.5. I split the labeled passes 80/20, stratified by movement type (51,006 for training and 12,752 held out). For the second and third collections I refit on all 63,758 labeled passes and tuned nothing. I report F1 for **in**, false alarms per 1,000 non-crossings and missed crossings per 1,000 crossings.

3. Results

Table 3 shows all four test sets.

Test set	Crossings found	Missed	False alarms	F1	False alarms / 1,000	Missed / 1,000
First collection, held out	3,932 / 3,934	2	8 / 8,818	0.999	0.9	0.5
Hard straight passes	3,247 / 3,275	28	-	-	-	8.5
Second collection	998 / 999	1	16 / 41,327	0.992	0.4	1.0
Third collection	-	-	24 / 3,897	-	6.2	-

Table 3. Logistic regression baseline on the four test sets at the 0.5 threshold. A dash means the test set has no passes of that kind.

First collection. On the 12,752 held-out passes, the model misses 2 of 3,934 crossings and gives 8 false alarms out of 8,818 turn-backs and parked tags. That is near perfect partly because every held-out **in** pass follows the forward rule. The hard straight passes are the real test: the rule rejects all of them, yet the model finds 99.1%. The largest weights are the middle-minus-ends gap (+7.5), outside heard (+6.2) and outside last heard (-5.1).

Second collection. Trained only on the first collection, the model finds 998 of 999 straight passes (F1 0.992), including 140 of the 140 that the forward rule would have rejected, with 16 false alarms out of 41,327 parked tags.

Third collection. All tags are parked, so only false alarms count: 24 of 3,897, mostly tags sitting close to the middle antennas (Appendix B).

4. Discussion

What I learned. Simple features that compare antenna groups separate crossings from parked tags, carry over to new recordings without retraining, and a linear model is enough. For a site: use the pattern across all antennas, not one strong read, and pick the threshold from what a false alarm costs against a missed crossing.

Were the goals met? Partly. The **in** vs. **none** task works on new recordings, but I haven't met the full goal yet, and the plan has changed since my proposal:

- **No outward crossings.** None of the collections have an **out** pass, so I haven't evaluated the three-class task or its macro-F1 yet.
- **No real turn-backs.** New recordings only have parked tags as negatives, which are easy.
- **Pass boundaries were given.** A live reader has to group its read stream into passes itself.
- **One split.** I planned 5-fold cross-validation; so far there is one 80/20 split.
- **Assumed port map.** The third collection's second antenna configuration uses ports 1-8, and I assumed they sit where ports 9-16 sit.

What I'd do differently, and next. I should have asked my advisor earlier about data with real turn-backs and outward passes. Next: results per tag type, live read grouping, and asking my advisor about turn-back and outward-pass data (Oct 26 - Nov 6); gradient-boosted tree, neural network and sequence models with ablations (Nov 9 - 20); robustness tests (Nov 23 - Dec 4); final held-out test (Dec 7 - 11).

5. Statement of Contributions

Ajin Frank Justin (sole author): data processing and labeling, exploratory analysis, feature engineering and selection, modeling, evaluation, presentation and report writing.

References

[1] Want, R. (2006). An introduction to RFID technology. *IEEE Pervasive Computing*, 5(1), 25-33. https://doi.org/10.1109/MPRV.2006.2

[2] Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.

[3] McKinney, W. (2010). Data structures for statistical computing in Python. *Proceedings of the 9th Python in Science Conference*, 56-61.

[4] Project data: first, second and third RFID read collections (unpublished, provided for this project, 2026).

Appendix A. Features and model weights

Here are all 19 features with the baseline's standardized weights from the 80/20 training split. A weight is the change in the log-odds of **in** for one standard deviation of the feature. A group that never heard the tag counts as -85 dBm in the signal gaps.

Feature	Definition	Weight
`read_outside`	1 if the outside group heard the tag	+6.19
`read_middle`	1 if the middle group heard the tag	-4.32
`read_inside`	1 if the inside group heard the tag	-3.04
`share_outside`	outside reads / all reads	-0.73
`share_middle`	middle reads / all reads	+2.21
`share_inside`	inside reads / all reads	-1.23
`first_pos_outside`	first round the outside group heard the tag / pass length in rounds	+0.72
`first_pos_middle`	first round the middle group heard the tag / pass length	+1.48
`first_pos_inside`	first round the inside group heard the tag / pass length	+4.09
`last_pos_outside`	last round the outside group heard the tag / pass length	-5.07
`last_pos_middle`	last round the middle group heard the tag / pass length	-4.14
`last_pos_inside`	last round the inside group heard the tag / pass length	-1.53
`peak_pos_outside`	round of the outside group's strongest read / pass length	-0.82
`peak_pos_middle`	round of the middle group's strongest read / pass length	+1.45
`peak_pos_inside`	round of the inside group's strongest read / pass length	+1.39
`max_mid_minus_edge`	strongest middle RSSI minus the stronger of the outside and inside maxima (dB)	+7.46
`p90_p10`	90th minus 10th percentile of all RSSI in the pass (dB)	+1.08
`max_in_out_gap`	absolute gap between the strongest inside and outside RSSI (dB)	+1.33
`n_mid_ports`	number of the 4 middle antennas that heard the tag	+4.82

Appendix B. Breakdowns

Hard straight passes	Passes	Found
looks backward	627	627 (100.0%)
middle only	2,648	2,620 (98.9%)

Table B1. Hard straight passes found, by why the forward rule rejected them.

Antenna configuration	Tag pose configuration	Parked tags	False alarms
0	0	1,227	7
0	1	1,140	6
1	0	770	6
1	1	760	5

Table B2. Third collection false alarms by configuration. Configuration 1 uses the assumed port map (port p taken to sit where port p + 8 sits).

Appendix C. Code

All my code is in the GitHub repository (https://github.com/justin-aj/ds5500-capstone). I set up the pipeline as marimo notebooks tracked with DVC: `data_ingestion.py`, `data_labeling.py` (the forward rule), `eda.py`, `feature_engineering.py`, `feature_selection.py` and `baseline_model_training.py`. Data processing uses pandas [3].